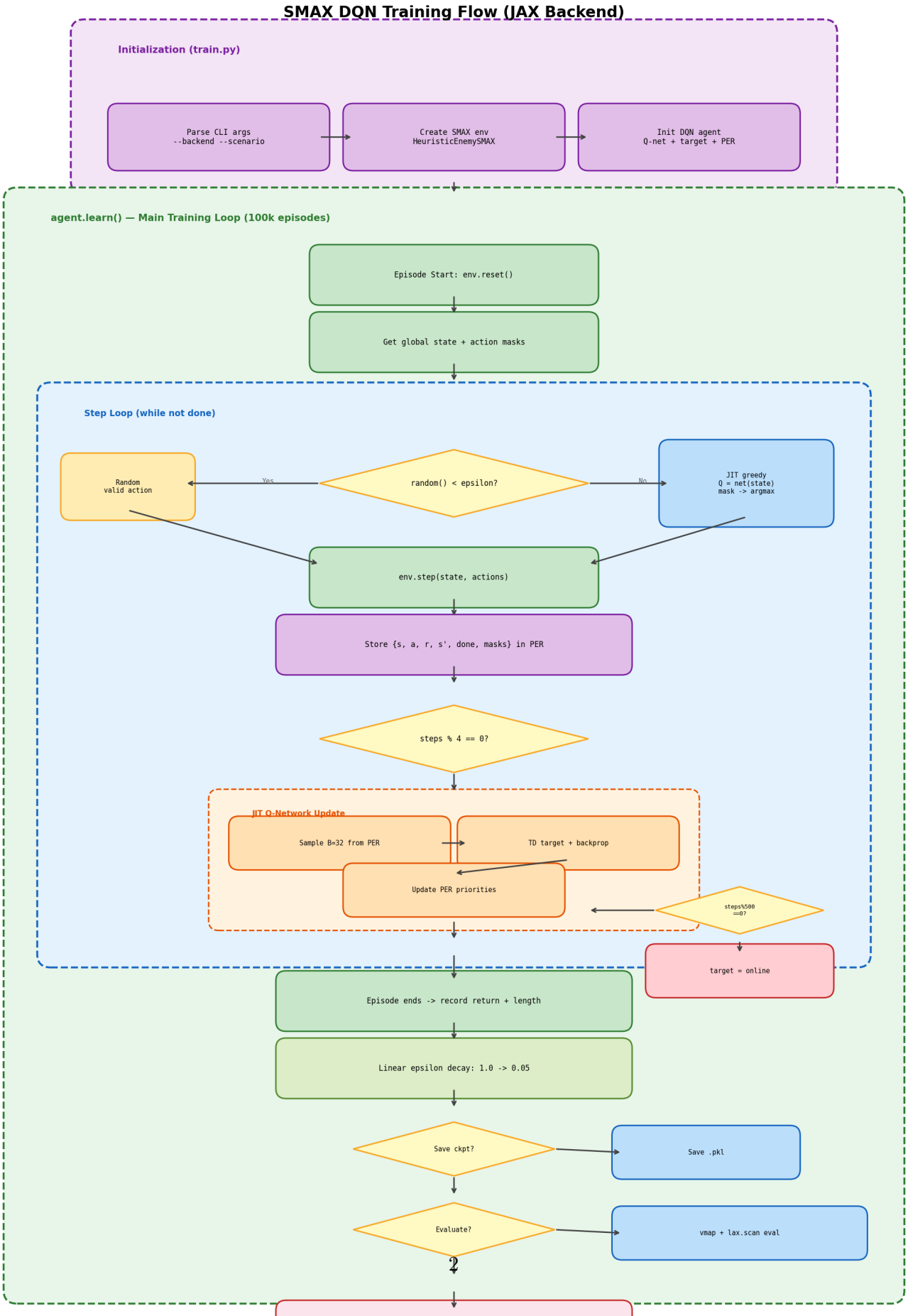
SMAX DQN Training Architecture (JAX Backend)

Overview

This system trains a **centralized DQN** agent to play StarCraft micromanagement scenarios (SMAX) using JAX. A single neural network takes the full global state and outputs Q-values for every agent simultaneously.

Training Flow



Neural Network Architecture

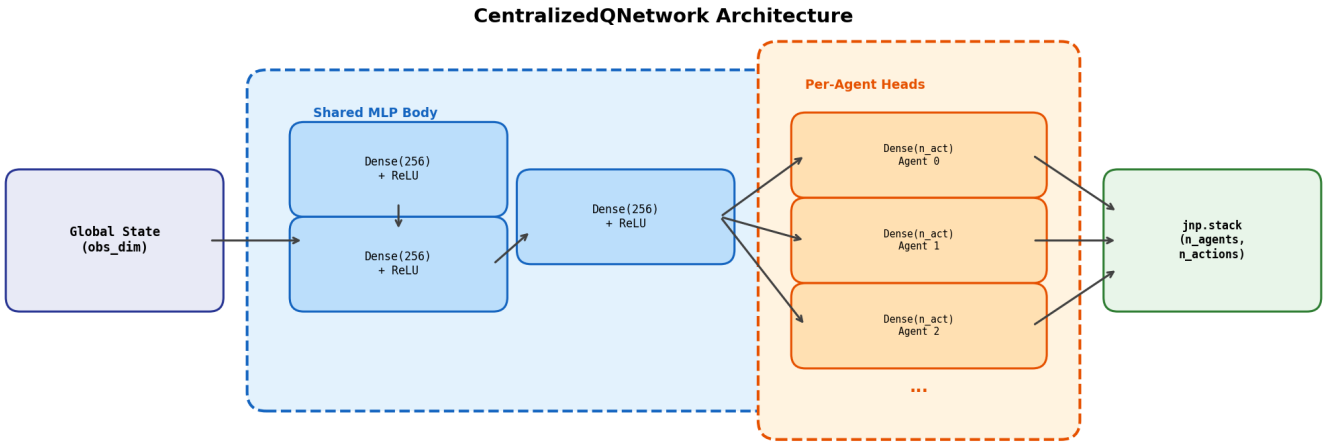


Figure 2: CentralizedQNetwork Architecture

Key design: Centralized Training with Decentralized Execution (CTDE). The network sees the full world state during training but outputs independent Q-values per agent, so each agent can act using only its own Q-head at inference time.

Prioritized Experience Replay (PER)

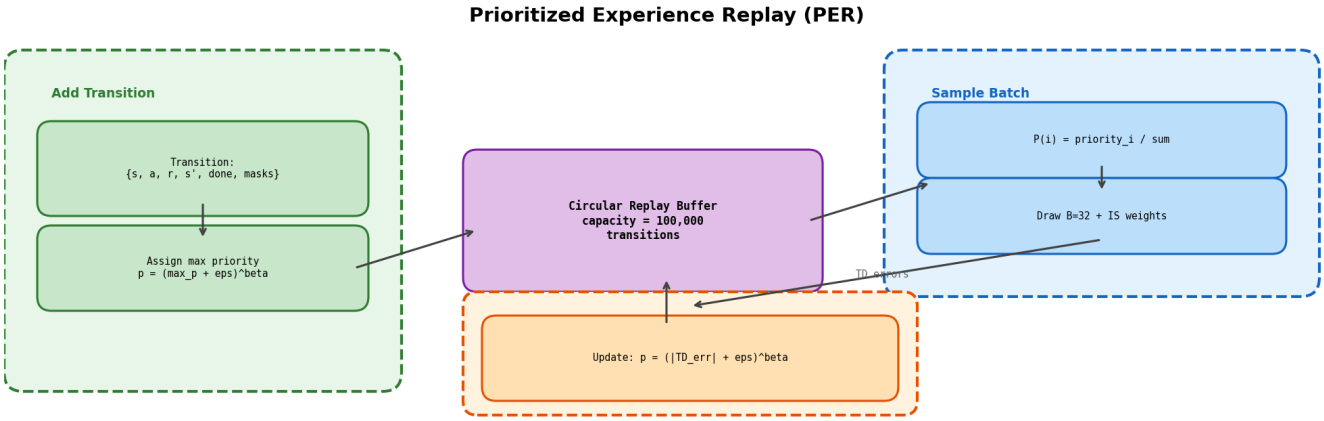


Figure 3: PER Diagram

Transitions with larger TD errors are sampled more frequently. Importance sampling weights correct the bias so the gradient updates remain unbiased.

File Map

File	Role
`train.py`	Entry point — CLI args, env creation, orchestrates training
`config.py`	Default hyperparameters
`buffers.py`	`PrioritizedReplayBuffer` — add, sample, update priorities
`utils.py`	Env helpers: `create_smax_env`, `get_global_state`, `evaluate`, plotting
`dqn_jax/dqn.py`	`DQN` class — learn loop, select_action, JIT update, save/load
`dqn_jax/policies.py`	`CentralizedQNetwork` — Flax MLP with per-agent heads
`train_smax_dqn.sh`	SLURM submission script — GPU allocation, CUDA libs, launches train.py

Key Hyperparameters

Parameter	Value	What it does
`gamma`	0.9	Discount factor (how much future rewards matter)
`epsilon`	$1.0 \rightarrow 0.05$	Exploration rate, decays linearly over 20k episodes
`alpha`	0.0005	Adam learning rate
`hidden_dim`	256	Width of MLP hidden layers (3 layers)
`M`	100,000	Replay buffer capacity
`B`	32	Batch size per gradient step
`C`	500	Steps between target network hard updates
`n_steps_for_Q_update`	4	Steps between Q-network gradient updates